



Using Bitwise Operators Instead of Logical Operators

Some bad programming practices become established as habits over time. Here's an example of one such bad practice.

In the last few years, I have come across many code segments in which bitwise operators were used instead of logical operators. I thought it was a mistake that only novice programmers make. But recently, while reading the book, *Software Engineering* by Ian Sommerville^[1], I found that he has made the same mistake. His book is well respected and is one of the most widely read on the subject. So I decided to write about this problem this month.

Consider the following Java code (from Ian's book):

```
if (n < 0 | n%2 == 1)
    throw new NumericException();
```

This code is a straightforward code: it is to ensure that n is a positive even integer—if n is some other value, a *NumericException* is thrown. In this code, the `|` (bitwise-OR) operator is used instead of the `||` (logical-OR) operator; the condition should have been this: $(n < 0 || n\%2 == 1)$.

In my experience, I have seen that most programmers who use `|` instead of `||` also use the `&` instead of the `&&` operator for Boolean operands. In fact, as I had guessed, in the subsequent few pages in the book, there was an expression in an assert statement that used `&` instead of the `&&` operator! In other words, for many programmers, using bitwise operators instead of logical operators becomes a (bad) habit.

Coming back to the code; yes, it will work: if $n < 0$ or $n\%2 == 1$ is true, then the *if* condition will evaluate to *true* and hence the exception will be thrown. So, why bother? To answer that, we'll have to understand some more details about the behaviour of bitwise and logical operators.

First, let us compare the truth table for the `|` and `||` operators:

A	B	A B	A B
T	T	T	T
T	F	T	T
F	T	T	T
F	F	F	F

Here, A and B are Boolean values, and T and F stands for *True* and *False*, respectively. The results for `A | B` and `A || B` look exactly the same! Yes, it's true, and that's why programs that make use of `|` instead of `||` for Boolean operands mostly work correctly. But why is it only 'mostly'?

The answer is that the truth table is not entirely correct.

[1] *Software Engineering*, 8th Edition, Ian Sommerville, Addison Wesley, 2006

For logical operators, there is a type of behaviour called 'short-circuiting': the expression is evaluated only till the truth value of the whole expression is known. Here, when A is true, no matter what the value of the other argument (B) is, the result for the `||` operator is true. So, B is a 'don't care' value and will not be evaluated. Whereas, for bitwise operators, the whole expression is always evaluated.

As you can see, this difference can cause subtle bugs in code. Here is a code segment (from real-world software) that has a serious bug:

```
if (container != null & !container.isEmpty())
    return container.getLength();
```

Here, when *container* is null, the second argument (`!container.isEmpty()`) is still executed, which would result in a null pointer access exception. Here, `&&` is the correct operator to use: in which case, when the container is null, the expression becomes false and hence the second argument will not be executed.

Programmers use bitwise and logical operators interchangeably for Booleans for various reasons. One main reason is efficiency: bitwise operators execute faster (doing bit-manipulation in CPU registers, which is faster) than logical operators (which internally perform a control transfer, which is slower). For most programmers, using such operators interchangeably creates confusion, and hence it's best to avoid this practice for reasons of efficiency—unless, for instance, it is embedded code where efficiency is very important. Another reason is ignorance: many programmers don't understand such subtle differences between the bitwise and logical operators. If the programmers come with a background in a 'non C-based' language, it is possible that they might make this mistake (many high-level languages do not support bitwise operators, so they use it interchangeably for Booleans).

We'll give Ian Sommerville the benefit of doubt: he might have a background steeped in some other language. However, as programmers, we should be careful about the programming practices we follow since many of them become habits later. 

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.